FlowRT

High-Performance Inference Engine for Iterative Generative Models

C++17 · CUDA 12.x · CUTLASS 3.x · ONNX Runtime · pybind11
Diffusion Policy · FLUX.1

PROJECT THESIS
FlowRT exploits three structural properties of flow matching that general inference engines ignore —
trajectory correlation, deterministic dynamics, and time-varying activations — and each maps to a concrete system contribution: a persistent kernel, a speculative sampler with formal bounds, and time-conditioned quantization.

Mohamed Tarabishy · Georgia Institute of Technology · Ongoing

What Is This Project? (Plain English)

Before diving into the technical spec, here's the entire project explained simply — what the problem is, why it matters, and what we're actually building.

The Problem, Starting From Scratch

What is a generative AI model?

A generative AI model is a machine that takes some input — a description, an image, a robot sensor reading — and produces an output — text, a picture, a robot action. The model is just a giant function: input goes in, output comes out.

What does 'inference' mean?

Inference is the word engineers use for 'running the model.' Training is when you teach the model. Inference is when you actually use it. This project is entirely about inference — making a trained model run faster.

What is flow matching?

Imagine you have a jar of mixed paint — complete noise, totally random. A flow matching model is trained to **gradually clean up that noise into something meaningful** over a series of small steps. Think of it like a sculptor starting with a rough block and making 50 passes with a chisel, each pass refining the shape a little more.

Step 1: random noise → slightly less noise. Step 2: slightly less noise → a bit more structured. Step 3 → 4 → ... → Step 50: finished output. Each step runs the entire model once. So 50 steps = 50 full passes through the model.

This is very different from how **LLMs (like ChatGPT) work**. LLMs generate one word at a time — each step produces a new token. Flow matching generates **the entire output at once, but refines it step by step**. Different structure. Different bottlenecks.

Why is flow matching slow?

Each of those 50 refinement passes runs the full model — which might be hundreds of millions of calculations on a GPU. On a modern GPU, a single pass might take 2-3 milliseconds. 50 passes = 100-150ms total. That's too slow for real-time robotics, which needs a decision every 10ms.

The existing tools that speed up AI inference (TensorRT, vLLM) were designed for a single forward pass or word-by-word text generation. They don't understand that flow matching is doing 50 correlated passes over a trajectory. They treat each pass as independent — which wastes a huge amount of opportunity.

The Core Insight

The Key Observation

Flow matching has three structural properties that nobody has fully exploited in an inference engine:

1. TRAJECTORY CORRELATION — step 24 and step 25 produce very similar intermediate states.
   The GPU is throwing away useful data between steps that it could have kept in fast memory.

2. DETERMINISTIC DYNAMICS — unlike LLMs (which involve randomness at each token), flow matching follows a mathematically predictable path. This lets us prove when we can skip work.

3. TIME-VARYING ACTIVATIONS — the numbers flowing through the model look very different at step 1 vs step 50. Standard compression techniques assume they're the same. They're not.

What We're Building: Three Contributions

Contribution 1: The Persistent Kernel (Systems)

Every time you run a GPU computation, there's a small overhead cost just to **start the computation** — like starting a car engine. With 50 flow matching steps, you start the GPU 50 times and pay that overhead 50 times.

The fix: write a single GPU program that handles all 50 steps internally without stopping. The intermediate state (the half-refined output) stays in fast GPU memory between steps instead of being written out to slow memory and read back in. This is like doing all 50 chisel passes without ever putting the sculpture down.

This is the centerpiece of the C++/CUDA engineering work. The result is measurable: we track GPU cache hit rates before and after, and expect to improve from ~40% to >85%.

Contribution 2: Speculative Flow Matching (ML Algorithm)

Speculative decoding is a technique used in LLMs: a small, fast model **drafts** several tokens ahead, and the large model **verifies** them all at once. If the drafts are good, you get K tokens for the cost of roughly 1 large model pass.

We're doing the same thing for flow matching — but the math is different. LLMs can check drafts using token probabilities. Flow matching has no probabilities; it's deterministic. So we need to **derive a new acceptance criterion** based on how much the small model's velocity prediction deviates from the large model's.

The key insight: errors in early steps (step 1-10) are more dangerous than errors in late steps (step 40-50), because early errors compound across all remaining steps. So the acceptance threshold must be **tighter early and looser late**. We derive this threshold mathematically and validate it empirically.

This is the ML research contribution. If the math holds up, it's publishable on arXiv.

Contribution 3: Time-Conditioned INT8 Quantization (Systems + ML)

Quantization is the practice of using lower-precision numbers (INT8 instead of FP16) to make models smaller and faster. Standard quantization picks one **scale factor** per layer and uses it for every input.

For flow matching, this is wrong. The numbers flowing through the model at step 1 (all noise) are very different from step 50 (nearly finished output). One scale factor fits neither well. We instead calibrate **one scale per timestep bin per layer** — 10 bins, so 10× more calibration points. This recovers most of the quality lost by naive quantization.

What Model Are We Optimizing?

Diffusion Policy (Chi et al., 2023) is the primary target. It's the most widely used flow/diffusion model in robot learning research — essentially the standard baseline for robot manipulation tasks. It's open source, the weights are on HuggingFace, and crucially: nobody has built an optimized inference engine for it. That's the gap.

FLUX.1 is the secondary target — an open-source image generation model that uses flow matching. This broadens the project's relevance beyond robotics to the generative AI space.

Why This Matters for a Resume

The companies that hire ML systems engineers — Together AI, Fireworks, Anyscale, NVIDIA — evaluate candidates on whether they can reason about hardware, find non-obvious optimizations, and implement them correctly in C++/CUDA. This project generates exactly that evidence:

- **Persistent kernel:** proves you understand GPU memory hierarchy, kernel launch mechanics, L2 cache behavior
- **Speculative FM:** proves you can do original algorithm work and derive formal guarantees
- **Time-conditioned quantization:** proves you understand the model's mathematical structure, not just the API
- **Real benchmarks on real models:** not a toy — Diffusion Policy is used in actual robotics research

Every component is independently verifiable. An interviewer can look at the Nsight Compute screenshots, read the acceptance criterion derivation, or run the benchmark themselves.

Technical Specification

1. Core Thesis

LLM inference has vLLM, TensorRT-LLM, and llama.cpp — each built around a specific structural property of autoregressive generation. Iterative generative models — flow matching, DDPM, DDIM — have no equivalent. General engines like TensorRT treat them as N independent forward passes and optimize each in isolation. This misses everything interesting.

The Three Exploitable Properties

1. TRAJECTORY CORRELATION — adjacent steps x_t and x_{t+dt} are highly similar; intermediate state has exploitable L2/shared memory residency across steps.

2. DETERMINISTIC DYNAMICS — flow matching trajectories are deterministic ODEs;

this enables a principled draft-verify scheme with formal error bounds.

3. TIME-VARYING ACTIVATION STATISTICS — activation distributions shift as $t \in [0,1]$; a single INT8 calibration scale per layer is provably suboptimal.

Primary target: Diffusion Policy (Chi et al., 2023) — most widely deployed flow/diffusion model in robot learning, pure PyTorch, no optimized inference engine exists.

Secondary target: FLUX.1 — for generative AI market signal.

Control loop target: <10ms per sample on RTX 4090, enabling 100Hz robot control.

2. Technical Contributions

2.1 Persistent Trajectory Kernel

Exploits: trajectory correlation

Standard flow matching inference launches N separate CUDA kernels — one per timestep. Each launch incurs scheduling overhead, and intermediate state is written to global memory between steps, evicting L2 cache lines that will be immediately re-read.

The insight: the entire N-step trajectory is known at launch time. A single persistent kernel can loop across all N steps internally, keeping x_t and hot activations resident in L2 between steps and eliminating N-1 kernel launch overheads entirely.

```
// Standard: N kernel launches, global memory round-trips
for (int t = 0; t < N_STEPS; t++) {
    launch_kernel(model_forward, x_t, timesteps[t]);  // N launches
    // x_t written to global memory, evicts L2
}

// FlowRT persistent kernel: 1 launch, L2-resident intermediate state
__global__ void persistent_trajectory_kernel(
    float* __restrict__ x,          // [B, D] — stays L2-resident
    const float* timesteps,         // [N_STEPS] — preloaded
    const float* condition,         // [B, C_dim]
    ModelWeights weights,
    int N_steps
) {
    __shared__ float x_shared[BLOCK_DIM];
    __shared__ float weight_tile[TILE_SIZE][TILE_SIZE];

    for (int step = 0; step < N_steps; step++) {
        float t = timesteps[step];
        float v = model_forward_fused(x_shared, t, condition, weights);
        #pragma unroll
        for (int i = threadIdx.x; i < D; i += blockDim.x)
```

```
            x_shared[i] += v * dt;
        __syncthreads();
    }
    store_result(x, x_shared);  // Single global memory write
}
```

What to measure with Nsight Compute: L2 hit rate before/after (target >85% vs ~40% baseline), global memory transactions, kernel launch overhead elimination, SM occupancy across step loop.

2.2 Speculative Flow Matching

Exploits: deterministic dynamics

Speculative decoding for LLMs works via token probability ratios. Flow matching is deterministic — no token probabilities. A new acceptance criterion is required based on ODE trajectory deviation bounds.

The setup: draft model f_φ (~10% parameters) proposes K steps; target model f_θ verifies all K in ONE batched forward pass.

```
// Draft K steps
x_1^d = x_0 + f_φ(x_0, t_0) · dt
x_2^d = x_1^d + f_φ(x_1^d, t_1) · dt
...
x_K^d = x_{K-1}^d + f_φ(x_{K-1}^d, t_{K-1}) · dt

// Target verifies ALL K in one batched pass
[v_0^θ, ..., v_{K-1}^θ] = f_θ([x_0, x_1^d, ..., x_{K-1}^d], [t_0, ..., t_{K-1}])
```

Acceptance criterion: accept step k if $||v_\theta(x_k^d) - v_\varphi(x_k^d)||_2 < \varepsilon(t_k)$, where the time-dependent threshold is tighter at $t \approx 0$ (high curvature, errors compound) and looser at $t \approx 1$ (near-linear, errors are local).

Trajectory deviation bound: $||x_k^{true} - x_k^d||_2 \leq \Sigma_{j<k} \delta_j \cdot \Delta t \cdot e^{L \cdot (t_k - t_j)}$, where L is the Lipschitz constant of the velocity field. This gives the formal threshold $\varepsilon(t_k) = \varepsilon_{total} / (K \cdot \Delta t \cdot e^{L \cdot t_k})$.

Key ablation: uniform ε vs time-dependent $\varepsilon(t_k)$ — this validates the theoretical derivation empirically.

2.3 Adaptive ODE Step Scheduling (RK45)

Exploits: trajectory correlation + deterministic dynamics

Fixed timestep schedules allocate NFE budget uniformly. Flow matching velocity fields have high curvature early (t≈0) and are nearly linear late (t≈1). Dormand-Prince RK45 adaptive stepping allocates steps where the trajectory actually needs them — typically 8-15 steps vs 50 fixed, with provable error bounds.

```
class AdaptiveRK45Sampler : public Sampler {
    float step(Tensor& x, float t, float dt, float tol) {
        // 6 function evaluations (FSAL: 5 effective)
        Tensor k1 = model.forward(x, t);
        Tensor k2 = model.forward(x + dt*a[1][0]*k1, t + a[1][0]*dt);
        // ... k3 through k6 via Butcher tableau

        float err = (x5 - x4).norm() / tol;
        if (err > 1.0f) return dt * 0.9f * pow(err, -0.2f);  // reject
        return dt * 0.9f * pow(err, -0.2f);                   // accept + grow
    }
};
```

2.4 Time-Conditioned INT8 Quantization

Exploits: time-varying activation statistics

Standard PTQ calibrates one scale per layer across all inputs. For flow matching, activations at t=0.1 and t=0.9 have different distributions — a single scale is a lossy compression of a non-stationary signal.

```
# Calibration: one scale per layer per timestep bin
N_BINS = 10
for t_bin in range(N_BINS):
    t_samples = sample_timesteps_in_bin(t_bin, n=1000)
    activations = collect_activations(model, data, t_samples)
    for layer, acts in activations.items():
        scales[layer][t_bin] = compute_optimal_scale(acts)

# Inference: select scale based on current timestep bin
__global__ void time_conditioned_int8_gemm(
    int8_t* A, int8_t* B, float* C,
    float* scales, int t_bin, int layer_idx) {
    float scale = scales[t_bin * N_LAYERS + layer_idx];
    int32_t acc = int8_dot_product(A, B);    // CUTLASS INT8 tensor core
    C[idx] = (float)acc * scale;
}
```

Expected gains: 2× memory reduction (FP16→INT8), 1.5-2× throughput on Tensor Core hardware, <1% quality degradation vs FP16 baseline with time-conditioned scales vs ~3-5% with global scales.

2.5 Supporting CUDA Kernel Library

Each kernel exists to serve a specific measured bottleneck, not as demonstration of skill.

- **Fused LayerNorm + Linear + Time Embedding Injection:** flow matching injects timestep t at every layer. Standard impl: 3 global memory round-trips. Fused: 1.
- **FlashAttention-2 style fused attention:** O(N) memory vs O(N^2), tiled computation with online softmax in registers.
- **CUTLASS INT8 GEMM:** hand-tuned for Sm89 (Ada Lovelace / RTX 4090) with time-conditioned dequantization.
- **Batched Heun integrator:** 2nd-order ODE step (corrector/predictor), better accuracy than Euler for same NFE.

3. C++ Engine Architecture

Organized around three design decisions, not file structure.

Decision 1: Static Memory Allocation

All GPU memory allocated at engine initialization. Zero dynamic allocation during inference. The memory pool owns pre-sized tensor buffers covering every intermediate activation in the model graph.

```
class FlowRTMemoryPool {
    struct Slot { void* ptr; size_t capacity; bool live; };
    std::vector<Slot> slots_;
    cudaStream_t compute_stream_;
    cudaStream_t transfer_stream_;
public:
    TensorView acquire(size_t bytes, DataType dtype);  // O(1) — pointer bump
    void release(TensorView);
    // Ping-pong: compute step t+1 while preparing embedding for t+2
    void overlap(std::function<void()> compute, std::function<void()> transfer);
};
```

Decision 2: Sampler as First-Class Abstraction

Sampling algorithm is runtime-selectable without recompiling the model graph. Euler, Heun, RK45 adaptive, and speculative flow matching all implement the same interface.

```
class Sampler {
public:
    virtual Tensor sample(const Model& model,
                          const Tensor& condition,
                          const SamplerConfig& cfg) = 0;
};

class SpeculativeFlowSampler : public Sampler {
    Model draft_model_;
```

```
    float epsilon_total_;
    bool accept(const Tensor& v_target, const Tensor& v_draft, float t) const;
    Tensor sample(const Model& target, const Tensor& cond, ...) override;
};
```

Decision 3: ONNX Model Graph with Custom Op Registration

Export PyTorch model to ONNX. Register custom ops via ONNX Runtime custom op API. FlowRT works on any flow matching model exportable to ONNX — not just Diffusion Policy.

```
OrtCustomOpDomain* domain = /* FlowRT domain */;
domain->Add(new PersistentTrajectoryOp());
domain->Add(new FusedLNLinearTimeOp());
domain->Add(new TimeConditionedINT8GemmOp());

// Public API
FlowRT::Engine engine("diffusion_policy.onnx", FlowRT::Config{
    .device = 0,
    .dtype = FlowRT::DataType::FP16,
    .quantize = FlowRT::QuantMode::INT8_TIME_CONDITIONED,
    .sampler = FlowRT::Sampler::SPECULATIVE,
    .draft_model = "policy_small.onnx",
    .num_steps = 50,
    .draft_steps = 5
});
auto sample = engine.sample(condition);  // <10ms target
```

4. Repository Structure

```
FlowRT/
├── include/flowrt/
│   ├── engine.hpp              # Public API
│   ├── sampler.hpp             # Sampler interface + implementations
│   ├── memory_pool.hpp         # Static GPU memory management
│   ├── model_graph.hpp         # ONNX graph + custom op registry
│   └── quantization.hpp        # Time-conditioned INT8
│
├── src/
│   ├── engine.cpp
│   ├── model_graph.cpp
│   ├── memory_pool.cpp
│   ├── samplers/
│   │   ├── euler.cpp
│   │   ├── heun.cpp
│   │   ├── rk45.cpp            # Adaptive Dormand-Prince
│   │   └── speculative.cpp     # Speculative flow matching
│   └── kernels/
│       ├── persistent.cu       # CENTERPIECE — persistent trajectory kernel
│       ├── attention.cu        # FlashAttention-2 style
```

```
│       ├── ln_linear_time.cu   # Fused LayerNorm + Linear + time embed
│       ├── integrator.cu       # Heun + RK45 step kernels
│       └── quantize.cu         # Time-conditioned INT8 GEMM (CUTLASS)
│
├── python/flowrt/              # pybind11 bindings
├── benchmarks/                 # Latency, throughput, quality benches
└── tests/
    ├── correctness/            # Numerical equivalence vs PyTorch (1e-4)
    └── performance/            # Nsight-based regression tests
```

5. Benchmarks & Evaluation

The Pareto curve (quality vs latency) is the main result — not a single latency number. Every contribution is presented as a tradeoff, not just a speedup claim.

Projected Latency Benchmarks (RTX 4090, batch=1, Diffusion Policy)

Method	Latency	NFE	vs PyTorch	Notes
PyTorch naive (50 steps, Euler)	~120ms	50	1×	Baseline
TensorRT FP16 (50 steps)	~40ms	50	3×	Comparison
FlowRT: persistent kernel only	~25ms	50	~5×	C++ contrib
FlowRT: persistent + RK45 adaptive	~18ms	~12 eff.	~7×	Algo contrib
FlowRT: persistent + speculative FM	~12ms	~10 eff.	~10×	ML contrib
FlowRT: all + INT8 time-conditioned	~7ms	~10 eff.	~17×	Full system

Important Note

These are projected targets, not measured results.
The actual numbers from benchmarking ARE the contribution.
Report with variance, not just means. Measure on specific GPU SKU with Nsight.

Kernel-Level Benchmarks (Nsight Compute)

For each custom kernel, report:

- L2 hit rate vs cuDNN baseline
- Memory bandwidth utilization (GB/s vs theoretical peak)
- Arithmetic intensity (FLOP/byte) — demonstrate compute-bound vs memory-bound
- SM occupancy %

Key Ablation Table

Configuration	Latency	Success Rate	Notes
Baseline (PyTorch)	120ms	85%	—
+ persistent kernel	25ms	85%	No quality loss
+ RK45 adaptive	18ms	84%	Tiny degradation
+ speculative FM (ε uniform)	14ms	83%	Quality drops
+ speculative FM (ε time-dep.)	12ms	84.5%	Theory validated
+ INT8 global scale	8ms	81%	Quality drops
+ INT8 time-conditioned	7ms	84%	Quality recovered

The pattern tells the story: time-dependent ε and time-conditioned INT8 both recover quality that naive versions lose. That's the theoretical contribution validated empirically.

6. Technology Stack

Category	Tool	Purpose
Core Engine	C++17	Engine, memory management, sampler abstractions
Core Engine	CUDA 12.x	All kernels
Core Engine	CUTLASS 3.x	INT8 tensor core GEMM (Sm89 target)
Core Engine	ONNX Runtime	Model loading + custom op registration
Core Engine	TensorRT 10.x	FP16 baseline for comparison
Build	CMake 3.25+ / vcpkg	Build system + dependency management
Profiling	Nsight Systems	Timeline: identify bottlenecks first
Profiling	Nsight Compute	Kernel-level: L2 hit rate, bandwidth, occupancy
Profiling	nvtx	Custom annotations in persistent kernel loop

Category	Tool	Purpose
Python	pybind11	Python bindings — drop-in for PyTorch sampling
Python	PyTorch	Training, ONNX export, correctness reference
Models	Diffusion Policy	Primary benchmark target
Models	FLUX.1	Secondary benchmark target
Testing	GoogleTest / pytest	C++ unit tests + Python integration tests

Company Signal Per Component

- **Persistent kernel + Nsight Compute** → NVIDIA, any inference engineering team
- **Speculative FM with formal bounds** → Together AI, Fireworks, Groq — they think in these terms daily
- **CUTLASS INT8** → NVIDIA internally, TensorRT team
- **ONNX custom ops** → broad industry standard — portable signal
- **Diffusion Policy target** → Physical Intelligence, Figure, any robotics ML team
- **FLUX.1 target** → Stability AI, generative AI infra teams

7. Development Milestones

Each milestone is independently shareable. You never need to finish to have something impressive on your resume.

Milestone 1 — The Kernel Work (6-8 weeks)

Goal

Working persistent kernel + fused ops with measured benchmarks on Diffusion Policy AND FLUX.1.

Result: strong GitHub repo + blog post. Resume-worthy immediately.

Targeting both models from the start broadens narrative from 'robotics engine' to 'flow matching engine'.

1. CUDA fundamentals — write a GEMM from scratch, profile with Nsight Compute, understand L2/shared memory hierarchy. Read FlashAttention-2 paper fully.
2. Export Diffusion Policy to ONNX. Measure baseline latency. Identify top 3 bottlenecks with Nsight Systems.
3. Implement persistent trajectory kernel. Profile L2 hit rate before/after. Iterate until L2 hit rate >80%.
4. Implement fused LayerNorm+Linear+time injection. Verify numerical correctness vs PyTorch (tolerance 1e-4).

5. Heun integrator kernel. Full end-to-end benchmark vs PyTorch baseline and TensorRT on Diffusion Policy.
6. Export FLUX.1 to ONNX. Run persistent kernel + fused ops benchmark on FLUX.1 — produces second benchmark table with minimal extra work.
7. Write benchmark report covering both models. Push to GitHub with Nsight screenshots.

Milestone 2 — The Quantization Work (4-6 weeks)

Goal

Time-conditioned INT8 with quality recovery demonstrated on both models.

Result: second independent contribution. No training dependency — purely calibration + CUTLASS kernel work.

Faster path to a second demonstrable contribution than Speculative FM.

8. Measure activation statistics σ(t) across layers on Diffusion Policy. Produce Figure 1 (theoretical motivation — shows distributions shift across timesteps).
9. Implement calibration pipeline. Calibrate per-timestep-bin scales for both Diffusion Policy and FLUX.1.
10. CUTLASS INT8 GEMM with time-conditioned dequantization kernel.
11. Quality ablation: global scale vs time-conditioned on both models. Validate theory empirically.
12. Benchmark: latency + quality Pareto curve for INT8 variants. Update GitHub with new results.

Milestone 3 — The Algorithms Work (8-10 weeks)

Goal

Speculative flow matching with formally derived acceptance criterion.

Result: arXiv-ready contribution. Do the literature search before claiming novelty.

Note: requires draft model training — this is the reason it comes after Milestone 2.

13. Literature search first — verify the acceptance criterion for deterministic ODE-based speculative sampling is not already published. Position contribution clearly before writing any code.
14. Derive acceptance criterion. Work through ODE error bound in LaTeX — forces rigor, becomes the paper.
15. Train draft model (10% parameters of Diffusion Policy). Verify velocity field agreement with target.
16. Implement speculative sampler in C++. Measure first acceptance rates.
17. Implement time-dependent threshold $\varepsilon(t_k)$. Ablation vs uniform ε — this validates the theory.
18. RK45 adaptive sampler. Measure NFE reduction vs fixed schedule.
19. Full Pareto curve — all sampler + quantization combinations vs quality. Final benchmark table.

8. Resume Line

Resume Bullet (fill X, Y, Z with real numbers)
FlowRT — C++17, CUDA 12.x, CUTLASS 3.x, ONNX Runtime, pybind11

- Inference engine for flow matching models exploiting trajectory correlation, deterministic dynamics, and time-varying activation statistics
- Persistent CUDA kernel eliminates N-1 kernel launch overheads, improving L2 hit rate from ~40% to >85% across 50-step trajectories
- Speculative flow matching: formally derived ODE-based acceptance criterion with time-dependent threshold; [X]x speedup at <1% quality degradation
- Time-conditioned INT8 quantization: per-timestep-bin calibration scales; 2x memory reduction vs FP16 with [Y]% less quality loss than global INT8
- [Z]x end-to-end latency reduction on Diffusion Policy vs PyTorch baseline

9. The One-Sentence Thesis

Say this in every interview
"FlowRT exploits three structural properties of flow matching that general inference engines ignore — trajectory correlation, deterministic dynamics, and time-varying activations — and each maps to a concrete system contribution: a persistent kernel, a speculative sampler with formal bounds, and time-conditioned quantization."